

RESEARCH mullvad

RESEARCH mullvad

Parity-bar research into Mullvad VPN's full current (2025–2026) feature set, for a self-hosted WireGuard-based reimplementation. Every claim cited inline; access date 2026-06-25. All sources listed at the bottom.

1. DAITA — Defense Against AI-guided Traffic Analysis

Threat model. A passive on-path observer (ISP, network tap) cannot read encrypted WireGuard payloads, but CAN observe packet *sizes* and *timing*. Modern ML/DL website-fingerprinting attacks (Deep Fingerprinting, Robust Fingerprinting) classify which site/service/contact you are using purely from those size+timing traces. DAITA defeats that. [mullvad-daita-blog, mullvad-daita-page]

Core techniques (three pillars): [mullvad-daita-page]

1. **Constant packet sizes** — all packets sent over the tunnel are padded to one uniform size, eliminating size as a feature.
2. **Random background ("cover") traffic** — dummy packets are unpredictably interspersed in *both directions* (client→server and server→client), so an observer cannot tell whether the device is actively transmitting or idle.
3. **Data-pattern distortion** — fake packets reshape the burst/timing pattern so the trace no longer matches a fingerprintable template.

Built on maybenot (see §2). One maybenot instance runs at the client, a second at the VPN relay; both inject padding/blocking actions. [maybenot-github, maybenot-witwer]

DAITA v1 architecture (hardcoded state machines): [daita-pulls-blog]

- Client side: four hardcoded maybenot machines — (1) sends padding if no data sent in a random [1.5–9.5] s window (defeats NetFlow "collapse"/idle detection); (2) ensures an outgoing packet for every 3rd packet received; (3) same for every 5th packet received; (4) a probabilistic padding machine firing after randomized delays.
- Relay side: three machines per defense — a NetFlow machine, an Interspace-inspired padding machine, and a uniquely generated machine tuned to reduce classifier accuracy against Deep/Robust Fingerprinting.

- Philosophy: *randomized* defenses rather than strict constant-rate regularization (partly forced by event-aggregation/reporting latency in the WireGuard-Go integration).

DAITA v2 changes (dynamic, server-driven defenses): [daita-pulls-blog, cyberinsider-daita-v2]

- Defenses are no longer hardcoded in the client. The **relay selects from a database of defenses** and pushes the appropriate maybenot machines + **limits ("padding budgets")** to the client at connect time.
- **~50% lower bandwidth overhead** vs v1 (Nov 2024 evaluation).
- Defenses can be improved/rotated **server-side without shipping a client update**; periodic DB updates invalidate any adversary model trained on the previous defense set.
- Better mobile/low-power performance; now available on desktop, Android, iOS.

Self-host takeaway: DAITA = maybenot (Rust, MIT) integrated into a WireGuard-Go data path on BOTH endpoints. To reimplement you need: (a) a WireGuard-Go fork that reports per-packet send/recv events into maybenot and lets it schedule padding/blocking; (b) a uniform-MTU padding scheme; (c) a server that ships machine definitions + budgets to the client at handshake. The defense machines themselves are data (probabilistic state machines), not code — they can be authored/generated offline.

2. maybenot framework (the engine under DAITA)

[maybenot-github, maybenot-crates, maybenot-docsrs, maybenot-acm, maybenot-witwer]

- **What it is:** a general Rust framework for traffic-analysis defenses, MIT-licensed, repo `maybenot-io/maybenot` (crates.io `maybenot`; v2.x current). Academically peer reviewed — published at WPES '23 (ACM, doi 10.1145/3603216.3624953).
 - **Lineage:** based on Tor's **Circuit Padding Framework** (Perry & Kadianakis 2019), itself a generalization of **WTF-PAD** (Juarez et al.) website-fingerprinting defense.
 - **Execution model:** an instance repeatedly takes **events** describing the encrypted traffic (packet sent, packet received, etc.) and emits **0+ scheduled actions** — *send padding* or *block outgoing traffic* (with a timer). Logic is encoded as one or more **probabilistic state machines** ("padding machines"). The framework enforces **limits/budgets** that cap how much padding or blocking each machine may do.
 - **Deployment:** symmetric — one instance per endpoint (client + VPN server, or Tor client + relay, or HTTPS server). Used by Mullvad in DAITA via their WireGuard-Go integration. The maybenot org also ships `maybenot-ffi` (C ABI) and a simulator.
 - **Self-host takeaway:** this is directly reusable. Embed the maybenot crate, feed it your tunnel's packet events, act on its padding/block actions. The hard research part (good defense machines) is partly solved by published machines + the simulator for evaluating new ones.
-

3. Multihop (entry/exit server separation)

[mullvad-multihop-help, mullvad-multihop-blog]

- **Mechanism:** WireGuard-in-WireGuard. Traffic is encrypted **twice on the client device** — an inner WireGuard tunnel to the *exit* server is sent inside an outer WireGuard tunnel to the *entry* server. Every Mullvad WG server is meshed to every other via server-to-server WireGuard tunnels, so any pair can act as entry+exit.
 - **Config trick (manual):** in the WG config, the **Endpoint port** selects the exit server and the **Endpoint IP/hostname** selects the entry server. The entry server port-maps that port to the chosen exit. An observer near the client sees only ordinary WG traffic to the entry server and cannot tell it will be forwarded, nor to where.
 - **Trust split:** entry server sees your source IP + which exit you chose, but not your traffic; exit server sees your traffic but only the entry server's IP, never your real IP. Correlating in/out flows across two different ISPs/hosters/jurisdictions is much harder; still secure even if the entry server is compromised (end-to-end encrypted).
 - **App config:** Advanced → VPN settings → WireGuard settings → Enable multihop, then pick entry + exit locations.
 - **Self-host takeaway:** needs ≥ 2 servers meshed with WG tunnels and a port→exit forwarding map on each entry server. The "double WG" approach is simpler to reimplement than a bespoke relay protocol.
-

4. Quantum-resistant tunnels (post-quantum WireGuard handshake)

[mullvad-pq-help, mullvad-pq-stable-blog, mullvad-pq-allservers-blog, mullvad-pq-intro-blog]

- **Why:** WireGuard's X25519 ECDH is harvest-now-decrypt-later vulnerable to a future quantum computer. PQ tunnels protect *today's* recorded traffic against *future* decryption.
- **How it works:** a normal WireGuard tunnel is established first, then used as a secure channel to negotiate a post-quantum **shared secret**; that secret is installed as WireGuard's **pre-shared key (PSK)** option (the WG PSK is mixed into the symmetric key schedule, so even if X25519 is broken the tunnel stays secure). The PSK is negotiated with an **ephemeral peer that is only temporarily valid** on the relay. Tool: `mullvad-upgrade-tunnel` (invoked via `WG PostUp = mullvad-upgrade-tunnel -wg-interface %i`). Success shows `preshared key: (hidden)` on the peer. (SaveConfig must be OFF or it overwrites the real config with the ephemeral peer.)
- **KEM algorithms** (selectable via `-kem` flag): [mullvad-pq-help]
 - `cme` — Classic McEliece 460896 Round3
 - `mlkem` — ML-KEM-1024 (NIST FIPS 203, formerly Kyber)
 - `cme-mlkem` — Classic McEliece + ML-KEM-1024 (**default**, hybrid)
 - `mlkem-cme` — ML-KEM-1024 + Classic McEliece The hybrid combines two independent KEMs so a flaw in either alone does not break the PSK. Mullvad moved off the 2017 "New Hope" experiment to NIST finalists.

- **Cost:** ~1–2 s extra to establish the shared secret; steady-state performance unchanged. PQ is now default/available on all WG servers.
 - **Self-host takeaway:** the protocol is "vanilla WG handshake → PQ KEM exchange over it → result becomes WG PSK." Classic McEliece has *huge* public keys (~half MB), which is why it is used for the static/server key and ML-KEM for the ephemeral part. Reusable building blocks: `liboqs` / Rust `ml-kem` + `classic-mceliece-rust`.
-

5. Obfuscation transports (anti-censorship / anti-DPI)

Mullvad now ships **four** WG obfuscation modes, all wrapping the WG UDP packets so DPI cannot fingerprint them as VPN traffic. An "auto/adaptive" mode tries them after several failed connection attempts. [mullvad-quic-blog, mullvad-lwo-blog, mullvad-shadowsocks-blog, privacyguides-obfs]

1. **UDP-over-TCP** (`udp2tcp`) — original method; tunnels WG's UDP inside a TCP stream to survive UDP-blocking firewalls. Higher latency (TCP-over-TCP meltdown risk). Open source as the Rust `udp-over-tcp` crate.
2. **Shadowsocks** — wraps WG in the Shadowsocks proxy protocol (an encrypted SOCKS-like stream designed to defeat the Great Firewall). Heavier CPU/throughput cost. Desktop + Android.
3. **QUIC obfuscation** — tunnels WG UDP through an HTTP/3 server via **MASQUE (RFC 9298, "Proxying UDP in HTTP")**. To a censor the traffic looks like ordinary HTTPS/HTTP-3 web traffic; blocking it would break the normal web. Keeps UDP speed. Desktop app 2025.9+. [mullvad-quic-blog]
4. **Lightweight WireGuard Obfuscation (LWO)** — newest (2025). Cheaply **scrambles the WireGuard packet header** so each packet is hard to fingerprint as WG, adding minimal overhead → best throughput + lowest power of the four (vs Shadowsocks). Desktop 2025.13+, Android 2025.9+; `mullvad obfuscation set mode lwo`. [mullvad-lwo-blog]

Self-host takeaway: `udp2tcp`, Shadowsocks, and QUIC/MASQUE all have mature open-source implementations you can put in front of a WG endpoint. LWO is the cheapest to add (a keyed header-scrambling pass on each packet) but is Mullvad-specific; MASQUE/QUIC is the strongest "blends with the web" option.

6. Encrypted DNS + content blocking

[mullvad-dns-malware-blog, mullvad-dns-adblock-blog, mullvad-doh-help, mullvad-dns-blocklists-gh]

- **Architecture:** every Mullvad VPN server runs its **own local DNS resolver**; all connected clients' queries egress behind that single server IP, so no individual's DNS queries are distinguishable. Free public encrypted DNS (DoH/DoT) is also offered to non-customers.
- **Blocking tiers** (separate resolver hostnames / Tailscale-style 100.64.0.x IPs):
 - `dns.mullvad.net` — vanilla, no blocking

- `adblock.dns.mullvad.net` — ads + trackers
 - `base.dns.mullvad.net` — ads + trackers + malware
 - `family.dns.mullvad.net` — + adult content
 - `extended` — + social media
 - `all.dns.mullvad.net` — ads + trackers + malware + adult + gambling + social media
 - **Transports:** DNS-over-HTTPS and DNS-over-TLS; macOS/iOS config profiles in `mullvad/encrypted-dns-profiles`. In-app toggles for each category.
 - **Data sources:** blocklists are open (`mullvad/dns-blocklists`); malware uses the **URLHaus RPZ** list imported into the WG/OpenVPN servers' resolvers.
 - **Self-host takeaway:** run a per-server resolver (`unbound/knot`) with RPZ blocklists; expose DoH/DoT; offer category sub-resolvers. All blocklist data + config are public.
-

7. Kill switch, lockdown mode, split tunneling

[[mullvad-killswitch-search](#), [mullvad-security-md](#), [mullvad-splittunnel-help](#)]

- **Kill switch** — **always on, cannot be disabled.** Implemented as "**fail closed**" **firewall rules**, not a separate process: in the *connecting* / *disconnecting* / *error* states the app installs firewall rules that block all traffic; if packets can't leave encrypted as intended, they can't leave at all. On connect, the app sets default routes `0.0.0.0/0` and `::/0` to force every flow through the tunnel.
 - **Lockdown mode** — extends fail-closed to the **disconnected** state: by default the disconnected state allows traffic; with lockdown on, disconnected behaves like the error state (blocks everything), so quitting/disconnecting the app leaves no internet until you reconnect or disable lockdown. Split-tunnel-excluded apps still get access.
 - **Split tunneling** — exclude specific apps from the tunnel; excluded apps use the real IP/regular connection (useful for VPN-blocking sites, local devices). Excluded apps keep internet even under lockdown mode. Platform mechanism differs per OS (cgroup/ netns on Linux, per-app on Android, etc.).
 - **Self-host takeaway:** kill switch is pure firewall policy (nftables/pf/WFP) gated on tunnel state — no special protocol. Split tunneling is OS-specific routing/marking.
-

8. Anonymous account-number model + no-logging architecture

[[mullvad-nolog-help](#), [mullvad-accountnumbers-blog](#), [threads-mullvad-summary](#)]

- **Account = one random number, nothing else.** Sign-up generates a random numbered account (no username, password, or email). The number + remaining time are the only identifiers. Numbers were lengthened over time for guessing resistance. Accounts are transferable/shareable; deliberately no usernames (a self-chosen name could leak identity/locale/cross-service correlation).
- **Payments decoupled from identity.** Accepts cash (envelope + payment token, mailed, envelope destroyed), Monero, Bitcoin/BCH, bank wire, card, PayPal,

Swish. Subscriptions were *removed* to avoid storing recurring payment data — time is sold in flat blocks. Crypto receiving addresses + transaction IDs deleted after 20 days (except Monero tx_hash).

- **No-logging (audited):** NO logging of traffic, DNS requests, connection/disconnection events or any timestamps, IP addresses, or per-user bandwidth.
 - **Minimal retained data:** per account — random number, expiry date, and (if used) the WireGuard public key + assigned tunnel address. Connection-count enforcement (5 simultaneous/account) is done **in RAM only, never persisted to disk** — so the limit works without a connection log.
 - **Diskless / RAM-only servers:** Mullvad runs servers diskless (System Transparency / stboot network-boot; nothing written to persistent disk), so a seized server yields no stored user data.
 - **Self-host takeaway:** the privacy guarantee is *architectural*, not policy-only — (a) identifier is a random token, not PII; (b) auth state (keys, sim-conn counts) lives in RAM; (c) servers boot diskless; (d) payment is decoupled. Reimplementable: random account tokens, in-memory session/key store, ephemeral/network-boot server images.
-

Honest gaps / unknowns (per §11.4.6 — stated, not guessed)

- The exact wire format of the DAITA v2 machine/budget push (client[?]relay) is not in public docs; only that the relay selects from a DB and sends machines + limits.
 - The precise multihop port→exit forwarding table format on entry servers is not publicly documented (the official multihop help page is deliberately high-level); the port = exit config convention is documented, the server-side mapping is not.
 - The PQ ephemeral-peer handshake message sequence beyond "establish WG → KEM exchange → install PSK via mullvad-upgrade-tunnel" is not fully specified in public docs.
-

Sources verified

- mullvad-daita-blog — <https://mullvad.net/en/blog/introducing-defense-against-ai-guided-traffic-analysis-daita> — accessed 2026-06-25
- mullvad-daita-page — <https://mullvad.net/en/vpn/daita> — accessed 2026-06-25
- cyberinsider-daita-v2 — <https://cyberinsider.com/mullvads-daita-v2-brings-stronger-resistance-to-ai-enhanced-vpn-traffic-analysis/> — accessed 2026-06-25
- daita-pulls-blog — <https://pulls.name/blog/2025-03-27-daita-v1-and-v2-defenses/> — accessed 2026-06-25
- maybenot-github — <https://github.com/maybenot-io/maybenot> — accessed 2026-06-25
- maybenot-crates — <https://crates.io/crates/maybenot> — accessed 2026-06-25
- maybenot-docsrs — <https://docs.rs/maybenot/latest/maybenot/> — accessed 2026-06-25
- maybenot-acm — <https://dl.acm.org/doi/10.1145/3603216.3624953> — accessed 2026-06-25

- maybenot-witwer — <https://www.ethanwitwer.com/posts/maybenot-framework/> — accessed 2026-06-25
- mullvad-multihop-help — <https://mullvad.net/en/help/multihop-wireguard> — accessed 2026-06-25
- mullvad-multihop-blog — <https://mullvad.net/en/blog/wireguard-multihop-now-easy-available-app> — accessed 2026-06-25
- mullvad-pq-help — <https://mullvad.net/en/help/quantum-resistant-tunnels-with-wireguard> — accessed 2026-06-25
- mullvad-pq-stable-blog — <https://mullvad.net/en/blog/stable-quantum-resistant-tunnels-in-the-app> — accessed 2026-06-25
- mullvad-pq-allservers-blog — <https://mullvad.net/en/blog/post-quantum-safe-vpn-tunnels-available-on-all-wireguard-servers> — accessed 2026-06-25
- mullvad-pq-intro-blog — <https://mullvad.net/en/blog/introducing-post-quantum-vpn-mullvads-strategy-future-problem> — accessed 2026-06-25
- mullvad-quick-blog — <https://mullvad.net/en/blog/introducing-quick-obfuscation-for-wireguard> — accessed 2026-06-25
- mullvad-lwo-blog — <https://mullvad.net/en/blog/introducing-lightweight-wireguard-obfuscation> — accessed 2026-06-25
- mullvad-shadowsocks-blog — <https://mullvad.net/en/blog/introducing-shadowsocks-obfuscation-for-wireguard> — accessed 2026-06-25
- privacyguides-obfs — <https://discuss.privacyguides.net/t/how-to-choose-the-best-obfuscation-method-with-mullvadvpn/32227> — accessed 2026-06-25
- mullvad-dns-malware-blog — <https://mullvad.net/en/blog/adding-another-layer-malware-dns-blocking> — accessed 2026-06-25
- mullvad-dns-adblock-blog — <https://mullvad.net/en/blog/how-were-knocking-down-ads-and-tracking> — accessed 2026-06-25
- mullvad-doh-help — <https://mullvad.net/en/help/dns-over-https-and-dns-over-tls> — accessed 2026-06-25
- mullvad-dns-blocklists-gh — <https://github.com/mullvad/dns-blocklists> — accessed 2026-06-25
- mullvad-killswitch-search — <https://mullvad.net/en/help/using-mullvad-vpn-app> — accessed 2026-06-25
- mullvad-security-md — <https://github.com/mullvad/mullvadvpn-app/blob/main/docs/security.md> — accessed 2026-06-25
- mullvad-splittunnel-help — <https://mullvad.net/en/help/split-tunneling-with-the-mullvad-app> — accessed 2026-06-25
- mullvad-nolog-help — <https://mullvad.net/en/help/no-logging-data-policy> — accessed 2026-06-25
- mullvad-accountnumbers-blog — <https://mullvad.net/en/blog/mullvads-account-numbers-get-longer-and-safer> — accessed 2026-06-25
- threads-mullvad-summary — <https://www.threads.com/@jacaranda7/post/DRzPij5kbA2/> — accessed 2026-06-25